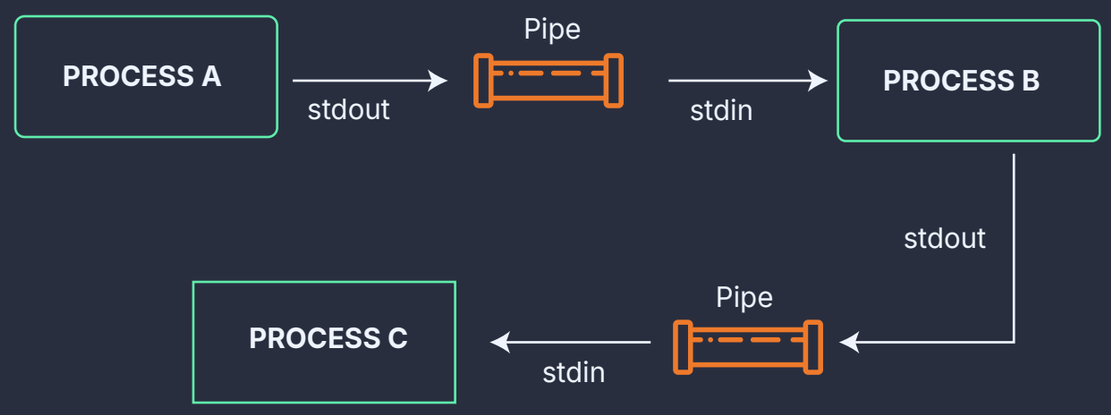
How Pipes Works in Linux?

Pipes allow inter-process communication by sending data from one process's output (stdout) to another's input (stdin).

Unidirectional: Data flows one way—from the writer to the reader.

Conceptual Flow: Process A (stdout) → [PIPE] → (stdin) Process B



Anonymous Pipes

A temporary, unidirectional communication channel between two related processes

Unidirectional: Data flows in one direction only.

Scope: Exists only during the execution of the processes.

Creation: Created programmatically using the pipe() system call or implicitly via the | operator in shell commands.

Usage: Typically used for chaining commands in a pipeline.

```
~$ cat file.txt | grep "keyword" | wc -l
```

`cat` reads the file.

Output is filtered by `grep`.

Filtered lines are counted by `wc`.

Named Pipe (FIFO)

Also known as FIFOs (First In, First Out).

A persistent communication channel that exists as a special file in the filesystem, allowing unrelated processes to communicate.

Unidirectional: Like anonymous pipes, data flows in one direction.

Persistence: Remains in the filesystem until explicitly removed.

Creation: Created using mkfifo or mknod

Scope: Can be accessed across different sessions or unrelated processes.

1. Create a named pipe:

```
~$ mkfifo my_pipe
```

2. Write to the pipe:

```
~$ echo "Hello, World!" > my_pipe &
```

3. Read from the pipe:

```
~$ cat < my_pipe
```

Important Notes

Anonymous pipes are ideal for simple, temporary tasks like chaining commands.

Named pipes are better suited for complex workflows requiring inter-process communication across unrelated processes or sessions.